

Historia de los lenguajes de programación



¿Qué es un lenguaje de programación?

Es una herramienta que permite desarrollar software o programas para computadora, mediante la creación e implementación de algoritmos de precisión que sirven como forma de comunicación entre la persona y la máquina.

Se conforma de una serie de símbolos y de reglas de sintaxis y semántica que definen su estructura y dan significado a sus elementos y expresiones.

Programar es el proceso de análisis, diseño, implementación, prueba y depuración de un algoritmo a partir de un lenguaje que compila y genera un código fuente ejecutable.

1

Tipos y estructuras de datos

Permiten manipular variables numéricas, de texto y otras más complejas como vectores, matrices y apuntadores.

2

Instrucciones

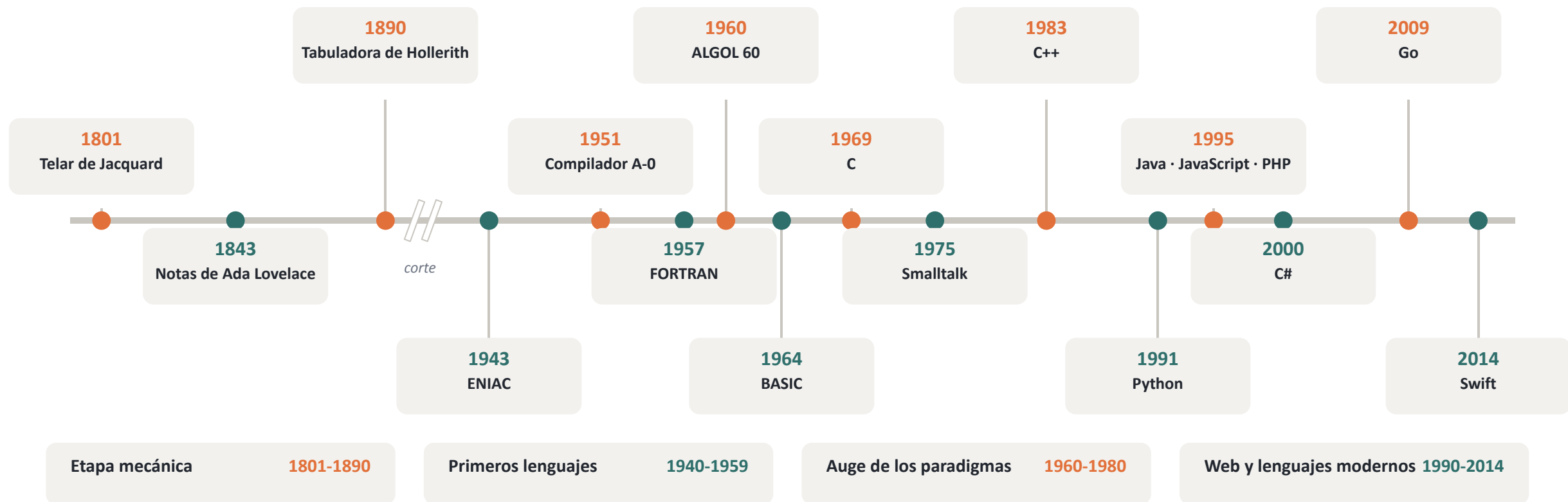
Estructuras gramaticales predefinidas, parecidas al lenguaje humano, que generan secuencias de acciones.

3

Control de flujo

Condicionales (if-then-else), bucles (for, while) o selección (case) que deciden la secuencia de acciones.

Línea de tiempo



Antecedentes

El telar programable de Jacquard

1801

El inventor francés Joseph Marie Jacquard creó tarjetas perforadas que se introducían en un telar; los orificios representaban los movimientos del brazo de la máquina y automatizaban patrones decorativos. Cada agujero funcionaba como una instrucción.

La máquina de diferencias de Babbage

1823

Con apoyo del gobierno británico se aprobó la construcción de un dispositivo mecánico pensado para realizar sumas de forma repetitiva. Babbage lo abandonó después para dedicarse a su máquina analítica.

La máquina tabuladora de Hollerith

1890

Herman Hollerith advirtió que la información podía codificarse en tarjetas perforadas y su tabuladora se empleó para procesar los datos del censo de Estados Unidos.

Charles Babbage y Ada Lovelace

Charles Babbage

Profesor de matemáticas e inventor en Cambridge a mediados del siglo XIX; fue el primero en concebir la idea de un lenguaje de programación y es considerado el padre de la informática.

Su máquina analítica —nunca construida por limitaciones tecnológicas— contaba con cinco unidades básicas: entrada, memoria, control, aritmético-lógica y salida.

Ada Lovelace

Entre 1842 y 1843 tradujo las memorias del matemático italiano Luigi Menabrea sobre la máquina propuesta por Babbage y añadió notas donde detallaba un método para calcular los números de Bernoulli.

Se le reconoce como la primera programadora de la historia: escribió el primer algoritmo destinado a ser procesado por una máquina.

Anticipó que las computadoras podrían manipular no sólo números, sino también símbolos e información.

Las bases teóricas de la computación

1

Alonzo Church

Expresó el cálculo lambda a través de fórmulas, mostrando que la lógica podía representarse con números y no sólo con palabras.

2

Alan Turing

En 1936 revolucionó la computación con la máquina de Turing, que demostraba que mediante un algoritmo se podía resolver cualquier problema matemático.

3

John von Neumann

La máquina de Turing sentó las bases para almacenar programas como datos en la arquitectura de von Neumann.

El código de Turing, a diferencia del cálculo lambda, no serviría satisfactoriamente como base para lenguajes de más alto nivel: su principal uso está en el análisis riguroso de la complejidad algorítmica.

Primeras Computadoras eléctricas

En la década de 1940 se crearon las primeras computadoras modernas con alimentación eléctrica. La velocidad y la capacidad de memoria limitadas obligaban a escribir programas en lenguaje ensamblador muy afinados; con el tiempo se comprobó que esa programación exigía un gran esfuerzo intelectual y era muy propensa a errores.

ENIAC

Programar cambiando cables

En la primera computadora, la ENIAC, la programación se basaba en componentes físicos: se cambiaban de sitio los cables y la entrada y salida de datos se hacía con tarjetas perforadas. En 1943 se creó su sistema de codificación.

1946-48

Plankalkül, de Konrad Zuse

El ingeniero alemán Konrad Zuse desarrolló este lenguaje —aplicado, por ejemplo, para jugar al ajedrez— y publicó un artículo sobre él en 1948, pero no se implementó en vida del autor y sus contribuciones quedaron aisladas.

1949

Short Code y códigos nemotécnicos

Surgió el short code, primer lenguaje aplicado en un dispositivo de cómputo electrónico, aunque traducido a mano. Entre 1949 y 1954 aparecieron conjuntos de instrucciones nemotécnicas, cada uno destinado a un fabricante específico.

Niveles

Lenguaje máquina

Sistema de códigos interpretable directamente por un circuito microprogramable. Los circuitos trabajan con dos niveles de tensión simbolizados con 0 y 1, y es específico de cada arquitectura.

Bajo nivel

Proporciona poca o ninguna abstracción del microprocesador, por lo que su traslado a lenguaje máquina es fácil. El ensamblador usa mnemónicos y traduce el archivo fuente a código máquina ejecutable.

Alto nivel

Su estructura semántica es muy similar a la forma en que escriben los humanos, lo que permite codificar algoritmos de manera más natural y expresar los programas de forma sencilla.

Escribir en lenguaje de máquina —instrucciones binarias comprensibles para las computadoras— era una tarea larga, tediosa, compleja y propensa a errores.

Los primeros compiladores y los lenguajes de alto nivel

1951

Grace Hopper y el compilador A-0

Trabajando para Remington Rand, comenzó el diseño del primer compilador ampliamente conocido, el A-0, liberado en 1957 con el nombre MATH-MATIC. El compilador permitió traducir instrucciones humanas a código binario.

1952

Autocode, de Alick E. Glennie

En la Universidad de Manchester concibió un sistema de programación llamado Autocode, un compilador muy rudimentario.

1957

FORTRAN, de John Backus

Sistema traductor de fórmulas matemáticas creado por un equipo dirigido por John Backus en IBM; primer lenguaje de alto nivel y primer compilador. Se diseñó para el cálculo científico y sigue en uso en la industria automovilística, gubernamental, aeroespacial y en la investigación.

1958

LISP, de John McCarthy

Destacó por su enfoque en el procesamiento simbólico y la inteligencia artificial.

1959

COBOL, de Grace Hopper

Creado durante su participación en el Short Range Committee, es un lenguaje uniforme para transacciones empresariales pensado para ser compatible con cualquier ordenador; está detrás de gran parte de los sistemas de transacciones de las empresas.

ALGOL 60

Un comité americano y europeo de científicos de la computación publicó el Reporte de ALGOL 60 (ALGO^rithmic Language), que consolidó muchas ideas que circulaban entonces.

ALGOL introdujo conceptos como las estructuras de control y la recursión, e influyó particularmente en el diseño de lenguajes posteriores.

Sus ideas continuaron en ALGOL 68, cuyas características poco usadas y su complejo sistema de coerciones automáticas de tipo lo hicieron impopular. Niklaus Wirth salió del comité de diseño para crear el sencillo lenguaje Pascal.

Estructuras de bloques anidadas

Las secuencias de código y sus declaraciones pueden agruparse en bloques sin pertenecer explícitamente a procedimientos separados.

Ámbito léxico

Un bloque puede tener variables, procedimientos y funciones propios, invisibles fuera de él: ocultamiento de información.

Notación BNF

La Backus-Naur Form describió la sintaxis del lenguaje; todos los lenguajes posteriores han usado alguna variante de BNF.

El auge de los lenguajes

APL

1962

Desarrollado por Kenneth E. Iverson.

Simula

1962

Superconjunto de Algol 60 creado por Nygaard y Dahl; primer lenguaje diseñado para apoyar la programación orientada a objetos e introducir clases y objetos.

SNOBOL

1962

Uno de los lenguajes de propósito específico del periodo.

CPL / BCPL

1963-67

Precursores del lenguaje C.

BASIC

1964

Creado por John Kemeny y Thomas Kurtz en Dartmouth College como lenguaje sencillo y accesible para principiantes. Una versión modificada por Bill Gates y Paul Allen fue el primer producto de Microsoft.

PL/I

1964

Lenguaje de propósito general de la época de los grandes ordenadores.

Florecimiento de paradigmas

C

1969-1973

Desarrollado por Dennis Ritchie y Ken Thompson en los Laboratorios Bell para el sistema Unix. Destacó por su eficiencia y por permitir programación de bajo nivel; gran parte del núcleo de Unix se reescribió en C y Linux sigue basado en él.

Pascal

1970

Creado por Niklaus Wirth, quien lo nombró en honor a Blaise Pascal. Se desarrolló como herramienta de enseñanza y formación, aunque su uso es comercial.

Prolog

1972

Diseñado por Colmerauer, Roussel y Kowalski; fue el primer lenguaje de programación lógica.

Smalltalk

Mediados de los 70

Proporcionó un diseño completo de lenguaje orientado a objetos y presentó el concepto de que todo es un objeto; se convirtió en base de muchos lenguajes modernos.

ML

1973

Construyó sobre Lisp un sistema de tipos polimórfico inventado por Robin Milner, pionero en los lenguajes funcionales con tipado estático.

El debate sobre la programación estructurada

«Programar sin usar GOTO»

El debate estaba estrechamente relacionado con el diseño del lenguaje: algunos no incluían GOTO, lo que obligaba al programador a estructurar el código. Hoy existe acuerdo casi general en que usarlo es una mala práctica.

Funciones

Con la programación estructurada surge el empleo de funciones: instrucciones situadas fuera del cuerpo principal del programa que realizan una tarea específica y devuelven un resultado, reutilizables una o varias veces.

Estructurada

Usa secuencias, decisiones y bucles; código organizado y fácil de seguir.

C, Pascal

Orientada a objetos

Organiza el código en objetos con propiedades y métodos.

Java, C++, Python

Funcional

Se basa en funciones puras y evita cambios de estado.

Haskell, Lisp

Lógica

Resuelve problemas mediante reglas y relaciones.

Prolog

Años de consolidación

C++

1980-1983

Bjarne Stroustrup modificó el lenguaje C en los Laboratorios Bell; combina la programación orientada a objetos con la programación de sistemas de bajo nivel. Está en el Top Ten de lenguajes desde 1986.

Ada

1983

El gobierno de Estados Unidos estandarizó este lenguaje de programación de sistemas destinado a contratistas de defensa.

Objective-C · Erlang

1986

Nuevas ramas de la orientación a objetos y de la concurrencia; Erlang fue creado por Joe Armstrong.

Perl

1987

Larry Wall, programador de Unix, lo diseñó como solución a las flaquezas del sistema; toma características de C, del shell sh, AWK, sed y Lisp.

Common Lisp · MATLAB · Eiffel

1984-1985

La comunidad de los lenguajes funcionales llevó a cabo la estandarización de ML y Lisp; Bertrand Meyer creó Eiffel.

RISC y compiladores

Década

El movimiento RISC postulaba que el hardware debía diseñarse para los compiladores más que para los programadores de ensamblador, lo que impulsó la tecnología de compilación.

Internet y la revolución de la Web

El rápido crecimiento de Internet fue el siguiente gran acontecimiento histórico para los lenguajes: al abrirse una plataforma totalmente nueva, se creó la oportunidad de adoptar lenguajes nuevos. La filosofía dominante fue la productividad del programador.

Haskell

1990

Lenguaje funcional de alto nivel desarrollado por un comité, basado en trabajos previos como ML y Miranda.

Python

1991

Creado por Guido van Rossum en el CWI de los Países Bajos; destaca por su legibilidad y simplicidad.

HTML · CSS

1991

Lenguaje de marcado para estructurar el contenido de las páginas web y hojas de estilo para definir su apariencia.

Ruby

1993

Yukihiro Matsumoto lo creó usando partes de sus lenguajes preferidos: Perl, Smalltalk, Eiffel, Ada y Lisp.

Java

1995

Desarrollado por Sun Microsystems para poner en marcha decodificadores de televisión interactiva; hoy está presente en más de mil millones de ordenadores.

JavaScript

1995

Creado por Brendan Eich en Netscape; se popularizó por su integración temprana con el navegador Netscape Navigator. Pese al nombre, no está relacionado con Java.

PHP

1994-1995

Rasmus Lerdorf lo desarrolló para reemplazar unos scripts de Perl de su web personal; hoy forma parte de millones de sitios web.

R · Delphi · Lua

1993-1995

R fue desarrollado por Ross Ihaka y Robert Gentleman en la Universidad de Auckland para análisis estadístico.

Lenguajes modernos y emergentes

2000-01

C#

Desarrollado por Microsoft y presentado públicamente en el año 2000 como parte de la plataforma .NET; se ha convertido en el lenguaje principal para aplicaciones de escritorio, web y móviles en esa plataforma.

2009

Go

Lanzado por Google; se inspira en C y destaca por su simplicidad y eficiencia, diseñado para la programación concurrente y los sistemas escalables.

2010

Rust

Lanzado inicialmente por Mozilla, es un lenguaje de sistemas reconocido por su enfoque en la seguridad, la concurrencia y el rendimiento.

2011-12

Kotlin

Desarrollado por JetBrains, se ejecuta sobre la máquina virtual de Java y Google lo señaló como el lenguaje óptimo para Android.

2014

Swift

Creado por Apple para aplicaciones iOS, macOS, watchOS y tvOS; destaca por su sintaxis moderna, seguridad de tipos e interoperabilidad con Objective-C.

Compiladores e intérpretes

Compilador

Traduce en tiempo de desarrollo: recibe todo el código fuente, lo analiza, lo optimiza y lo traduce a lenguaje máquina, dejando un programa completo listo para ejecutarse.

Al código objeto se le agregan las librerías mediante un enlazador (linker) y se obtiene el código ejecutable.

Ejemplos: C, C++, Pascal y Visual Basic.

Intérprete

Traduce en tiempo de ejecución: analiza y ejecuta el código fuente a medida que el programa avanza, leyendo la instrucción línea por línea y obteniendo el código máquina correspondiente.

Ejemplos: PHP, Perl, Python y Basic.

Unos programas especiales convierten las instrucciones escritas en código fuente en instrucciones escritas en lenguaje máquina, es decir, en ceros y unos.

Tendencias actuales y futuro

1

Programación funcional

Aumenta su soporte en lenguajes comerciales importantes, incluida la programación funcional pura, para hacer el código más fácil de razonar y de paralelizar.

2

Concurrencia y distribución

Construcción de lenguajes que apoyan la programación concurrente y distribuida, con mayor énfasis en distribución y movilidad.

3

Seguridad y confiabilidad

Mecanismos de verificación: chequeo sintáctico extendido, control de flujo de información y seguridad de hilos.

4

Código abierto

Filosofía de desarrollo de lenguajes que incluye la colección de compiladores de GNU y lenguajes como Python, Ruby y Squeak.

5

Inteligencia artificial

La programación es la base de la IA y el machine learning: permite diseñar algoritmos que entrenan a las máquinas para mejorar con la experiencia.

6

Herramientas sin código

El auge de la programación visual y las herramientas no-code busca que más personas desarrollen tecnología sin ser expertas en código.

Más de dos mil lenguajes

En la actualidad hay más de dos mil lenguajes de programación y cada día se crean otros que emplean de forma más eficiente los recursos de las computadoras y facilitan la tarea de programar. Es importante conocer sus tipos, características y plataformas, ya que según el trabajo a realizar se deberá elegir el más adecuado.

Fuentes consultadas

Wikipedia, «Historia de los lenguajes de programación» — es.wikipedia.org

Epitech, «Evolución de los lenguajes de programación: Inicio y actualidad» — epitech-it.es

UNAM-CUAED, «Lenguajes de Programación» (UAPA), Uriel Marín Monterde — repositorio-uapa.cuaed.unam.mx

Luis Llamas, «Evolución de los lenguajes de programación» — luisllamas.es

Superprof, «Historia de la Programación: Evolución de los Lenguajes de Programación» — superprof.mx